TECHNICAL DEEP-DIVE · TRANSFORMERS

The Final Output Layer

Linear Projection & Softmax in the Transformer

After all the attention and feed-forward computation, a Transformer must collapse its high-dimensional hidden state into a single probability distribution over its vocabulary. This report dissects the two-stage mechanism — the linear layer and the softmax function — that converts abstract representations into human-readable tokens.

AUTHOR

Uditya Narayan Tiwari

TOPIC

Transformer Output Head

PREREQUISITES

Decoder Architecture

DATE

June 2026

SECTION 01

Where Does the Output Layer Sit?

Every Transformer decoder stack — whether in a seq2seq model like the original Transformer or a decoder-only model like GPT — ends with the same two operations: a **linear projection** to vocabulary space and a **softmax normalization** to convert raw scores into a probability distribution. This tiny stage at the very end determines what word (or token) gets generated next.

d	|V|	50k+	1
MODEL DIM (INPUT)	VOCAB SIZE (OUTPUT)	TYPICAL VOCAB SIZE	TOKEN CHOSEN PER STEP

input

Embedding + Positional Encoding

tokens → vectors of size d_model

↓

N× blocks

Decoder Stack (N layers)

self-attn → cross-attn → FFN

↓

last hidden

Final Decoder Hidden State h_t

shape: [batch, seq_len, d_model]

focus here

↓

linear

Linear Layer $W \cdot h_t + b$

shape: [batch, seq_len, |V|] — logits

↓

normalize

Softmax → P(token | context)

shape: [batch, seq_len, |V|] — probs

↓

output

Next Token (argmax / sample)

one integer index into vocabulary

The Linear Projection Layer

The final hidden state h_t produced by the decoder has shape **[d_model]** per token — say 512 or 4096 dimensions depending on the model size. This is a dense vector that encodes everything the model knows about what should come next, but it has no direct correspondence to any word.

The **linear layer** (also called the "language model head" or LM head) is a single learned weight matrix **W** of shape *[d_model × |V|]* that projects this hidden vector into a raw score (logit) for every token in the vocabulary.

LINEAR PROJECTION

$$\text{logits} = W \cdot h_t + b$$

$W \in \mathbb{R}^{|V| \times d\_model}$ $h_t \in \mathbb{R}^{d\_model}$ $b \in \mathbb{R}^{|V|}$ (optional bias)

$\text{logits} \in \mathbb{R}^{|V|}$

The result is a vector of **|V| unnormalized scores** called logits — one for every token in the vocabulary. A high logit means the model assigns high "confidence" to that token being next; a low logit means the opposite. These are raw numbers, not probabilities — they can be negative, very large, or very small.

Weight Tying (Shared Embeddings)

An important and widely-used optimization: in most modern models, the linear layer's weight matrix **W** is *tied* (shared) with the input embedding matrix. Both have shape *[|V| × d_model]*, so they naturally match. Weight tying:

⚖️

Halves parameter count

Eliminates |V| × d_model parameters — e.g. 50,000 × 512 = 25.6M fewer params. Critical at scale.

🧠

Semantic consistency

Forces the model to embed tokens and score them in a shared semantic space — geometrically coherent representations.

Empirically better

Press & Wolf (2017) showed weight-tied models achieve lower perplexity on language modeling benchmarks.

PyTorch one-liner

`model.lm_head.weight = model.embed.weight` — simply assign the same tensor.

Scale note: For GPT-3 (d_model = 12,288, |V| = 50,257), the unshared LM head would be 12,288 × 50,257 ≈ **617 million parameters** — nearly 4× the entire GPT-2 model. Weight tying makes large vocab models tractable.

SECTION 03

The Softmax Function

Raw logits are not probabilities. They can be any real number. The **softmax function** takes the logit vector and converts it into a valid probability distribution — all values are positive and sum to exactly 1.0 — by exponentiating and normalizing.

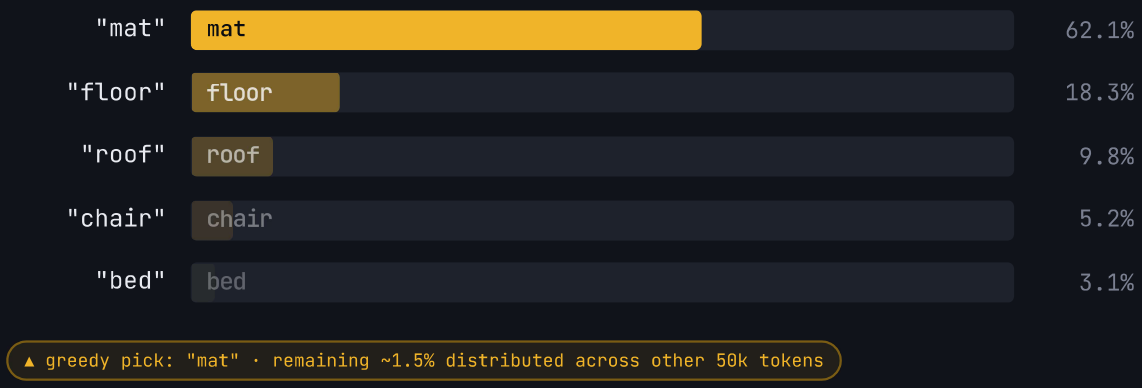
SOFTMAX DEFINITION

$$P(\text{token} = i \mid \text{context}) = \exp(z_i) \,/\, \Sigma_j \exp(z_j)$$

z = logit vector $\in \mathbb{R}^{|V|}$ | z_i = logit for token i | $\Sigma\, P(i) = 1$ (always) | $P(i) \in (0, 1)$ (always)

Exponentiation amplifies differences: a token with logit 5.0 vs one with logit 3.0 gets exp(5)/exp(3) = e^2 ≈ 7.4× more probability mass. This means small logit differences get magnified into large probability differences — softmax is "winner-takes-most" in nature.

EXAMPLE: SOFTMAX OVER TOP-5 TOKENS (CONTEXT: "THE CAT SAT ON THE ___")



Numerical Stability — Log-Sum-Exp Trick

Computing softmax naively can cause overflow: exp(logit) for large logits (e.g. logit = 1000) overflows float32 (max ≈ 3.4×10^{38}). The standard fix is to subtract the maximum logit before exponentiation — mathematically equivalent, numerically safe:

NUMERICALLY STABLE SOFTMAX

$$P(i) = \exp(z_i - \max(z)) \;/\; \Sigma_j \exp(z_j - \max(z))$$

`max(z) subtracted from each $z_i$` `largest exponent = exp(0) = 1.0`

`no overflow possible`

Temperature Scaling on Logits

Before applying softmax, logits are optionally divided by a **temperature T**. This is done at inference time to control the "sharpness" of the distribution:

TEMPERATURE-SCALED SOFTMAX

$$P(i) = \exp(z_i / T) \;/\; \Sigma_j \exp(z_j / T)$$

`T → 0 → argmax (greedy)` `T = 1 → default softmax`

```
T → ∞  → uniform distribution
```

Temperature	Effect on Distribution	Typical Use
T < 1 (e.g. 0.3)	Sharper — top token gets even more mass	Factual Q&A, code completion
T = 1.0	Default softmax — training distribution	Standard generation
T = 0.7–0.9	Slightly smoothed — common inference sweet spot	Chat, summarization
T > 1 (e.g. 1.5)	Flatter — more diversity, more risk of nonsense	Creative writing, brainstorming

SECTION 04

Training vs. Inference — Two Different Modes

The linear + softmax head behaves differently depending on whether the model is training or performing inference. Understanding this distinction is critical.

T

Training — Cross-Entropy Loss on Logits

During training, softmax is computed implicitly inside the cross-entropy loss function. PyTorch's `nn.CrossEntropyLoss` accepts raw logits — it applies log-softmax internally for numerical stability. The loss measures how much probability the model assigned to the correct next token.

I

Inference — Explicit Softmax + Sampling

During inference, the model produces logits, optionally scales by temperature, then applies softmax to get probabilities, then samples (or takes argmax) to pick the next token. The chosen token is appended to the sequence and the loop continues.

CROSS-ENTROPY LOSS (TRAINING OBJECTIVE)

$$L = -(1/N) \cdot \Sigma_n \log P(y_n \mid x_1 \ldots x_{n-1})$$

y_n = true next token at step n

P comes from softmax over logits

minimize L → maximize correct-token probability

Common mistake: Passing probabilities (post-softmax) into `nn.CrossEntropyLoss` in PyTorch is a bug — it expects raw logits. Similarly, applying softmax twice collapses the distribution incorrectly. Always feed logits directly to the loss function.

SECTION 05

Implementation in PyTorch

Below is a complete, annotated implementation of the LM head — linear projection, optional weight tying, temperature-scaled softmax, and both training loss and inference token selection.

PYTHON · PYTORCH

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class TransformerOutputHead(nn.Module):
    """
    The final two-stage output head of a Transformer:
      1) Linear projection  : h_t  [d_model] → logits [vocab_size]
      2) Softmax            : logits → probability distribution

    Supports weight tying with the input embedding matrix.
```

```
    """

    def __init__(
        self,
        d_model: int,
        vocab_size: int,
        bias: bool = False,       # GPT-2 style: no bias on lm_head
        tie_weights: bool = True,
    ):
        super().__init__()
        self.vocab_size = vocab_size

        # Input embedding: [vocab_size, d_model]
        self.embed = nn.Embedding(vocab_size, d_model)

        # LM head: [d_model] → [vocab_size]
        self.lm_head = nn.Linear(d_model, vocab_size, bias=bias)

        if tie_weights:
            # Share weight matrix between embedding and lm_head
            self.lm_head.weight = self.embed.weight

    def get_logits(self, hidden: torch.Tensor) -> torch.Tensor:
        """hidden: [B, T, d_model] → logits: [B, T, vocab_size]"""
        return self.lm_head(hidden)

    def training_loss(
        self,
        hidden: torch.Tensor,   # [B, T, d_model]
        targets: torch.Tensor,  # [B, T] — integer token ids
    ) -> torch.Tensor:
        """Cross-entropy loss. Passes raw logits — no manual softmax needed."""
        logits = self.get_logits(hidden)          # [B, T, V]
        B, T, V = logits.shape
        # CrossEntropyLoss expects [N, C] and [N] — flatten batch+time
        return F.cross_entropy(
            logits.view(B * T, V),
            targets.view(B * T),
        )

    @torch.no_grad()
    def next_token(
        self,
        hidden: torch.Tensor,   # [B, T, d_model]
```

```
        temperature: float = 1.0,
        top_k: int | None = None,
    ) -> torch.Tensor:
        """Inference: sample next token from the final position."""
        logits = self.get_logits(hidden)[:, -1, :]  # last position → [B, V]

        # Apply temperature scaling
        if temperature ≠ 1.0:
            logits = logits / temperature

        # Optional top-k filtering (zero out tail probabilities)
        if top_k is not None:
            v, _ = torch.topk(logits, top_k)
            logits[logits < v[:, [-1]]] = float('-inf')

        # Softmax → probabilities → multinomial sample
        probs  = F.softmax(logits, dim=-1)          # [B, V]
        token  = torch.multinomial(probs, 1)          # [B, 1]
        return token


# ── Quick test ──────────────────────────────────────
head = TransformerOutputHead(d_model=512, vocab_size=50257)
hidden = torch.randn(2, 16, 512)          # batch=2, seq=16
targets = torch.randint(0, 50257, (2, 16))

loss  = head.training_loss(hidden, targets)
token = head.next_token(hidden, temperature=0.8, top_k=50)

print(loss)     # tensor(10.82...)  — random init ≈ log(|V|)
print(token)    # tensor([[23917], [8042]])  — sampled token ids
```

SECTION 06

Modern Variants & Optimizations

The basic linear + softmax head has been refined significantly in modern LLMs. Here are the key variations found in production models:

Technique	Model(s)	What it does
Weight Tying	GPT-2, LLaMA, most LLMs	Shares embed ↔ lm_head weights; saves params, improves coherence
Pre-norm before head	LLaMA, Mistral	Applies RMSNorm to final hidden before linear projection; stabilizes training
No bias in LM head	GPT-2, GPT-3	Removes bias term from linear layer; reduces params, rarely hurts performance
Top-p (Nucleus) Sampling	GPT-3, Claude	Truncates vocab to smallest set summing to p before sampling; more adaptive than top-k
Repetition Penalty	Various	Scales down logits of already-generated tokens; reduces looping
Speculative Decoding	Gemini, LLaMA 3	Draft model generates k tokens; target model verifies in one forward pass; 2-3× speedup
Logit Processors	HuggingFace Transformers	Pipeline of modifiers (min-length, no-repeat-ngram, bad-words) applied to logits pre-softmax

Speculative Decoding insight: The softmax output is used not just to pick tokens, but to verify tokens in speculative decoding. The target model runs a single forward pass over k drafted tokens, checks their probabilities against acceptance thresholds, and accepts or corrects — achieving multi-token output per model invocation.

SECTION 07

Key Takeaways

The output head is deceptively simple — just a matrix multiply and an exponentiation — yet it is the stage where abstract computation becomes language. Every word, every token, every response a language model produces flows through these two operations.

Linear maps to vocabulary space

The weight matrix $W \in \mathbb{R}^{|V| \times d}$ is the Rosetta Stone — it maps every direction in hidden space to a token score.

Softmax normalizes + temperature shapes

Softmax enforces a valid probability distribution. Temperature is the primary knob for creativity vs. precision at inference.

Weight tying is almost always used

Sharing embed ↔ head weights is a nearly universal practice in modern LLMs — it reduces params and improves semantic consistency.

Training uses logits, not probs

Cross-entropy loss in PyTorch takes raw logits. Never apply softmax before the loss function — it's built in and numerically safer.

AUTHOR

Uditya Narayan Tiwari

Technical Report — Transformer Output Layer

LINKS

Portfolio

GitHub

LinkedIn

RESOURCES

Knowledge Base

Vaswani et al. (2017) · Press & Wolf (2017)

VERSION

1.0 — June 2026